Índice

1. Descripción del Taller **2**

2. Objetivos **2**
2.1. Objetivo general 2
2.2. Objetivos específicos 2

3. Marco Teórico **2**

4. Metodología **3**
4.1. Configuración del Hardware 3
4.2. Configuración del Software 3
4.3. Descripción del algoritmo 4

5. Implementación Secuencial en CPU **5**

6. Implementación Acelerada en GPU **5**

7. Resultados **6**
7.1. Resultados obtenidos en CPU 6
7.2. Resultados obtenidos en GPU 7
7.3. Comparación general de resultados 7

8. Conclusiones **8**

9. Repositorio o Evidencia de Ejecución **8**

1. Descripción del Taller

El presente taller tiene como objetivo desarrollar un algoritmo de procesamiento de video orientado a la detección automática y al seguimiento de la trayectoria de un asteroide sobre un fondo estelar.

El problema consiste en analizar una secuencia de video, identificar la presencia de un objeto en movimiento y reconstruir su recorrido a partir de las posiciones detectadas en cada frame. Para ello, se requiere implementar dos soluciones: una versión secuencial ejecutada en CPU y una versión acelerada mediante GPU, con el fin de comparar su comportamiento y analizar el impacto del procesamiento paralelo en este tipo de tareas.

La entrada del sistema corresponde a un archivo de video en formato `.mp4`. El procesamiento debe incluir limpieza de ruido, detección del objeto y cálculo de trayectoria. Como salida, el algoritmo debe producir un veredicto claro indicando si existe o no un asteroide en el video, así como una gráfica que represente la trayectoria detectada sobre el fondo estelar.

2. Objetivos

2.1. Objetivo general

Desarrollar e implementar un algoritmo de detección y seguimiento de trayectoria de asteroides en video, comparando una versión secuencial en CPU con una versión acelerada mediante GPU.

2.2. Objetivos específicos

- Leer y procesar un video astronómico en formato `.mp4`.
- Reducir el ruido presente en los frames para mejorar la detección.
- Detectar la presencia de un objeto en movimiento mediante diferencia temporal entre frames.
- Calcular el centroide del objeto detectado en cada frame.
- Construir la trayectoria del asteroide a partir de las coordenadas registradas.
- Comparar el tiempo de ejecución entre la implementación en CPU y la implementación acelerada en GPU.

3. Marco Teórico

El procesamiento de video puede entenderse como el análisis secuencial de una serie de imágenes digitales llamadas frames. Cada frame representa una instantánea del video en un instante de tiempo determinado. En problemas de detección de movimiento, una estrategia común consiste en comparar frames consecutivos para identificar variaciones espaciales en la escena.

En este taller, la detección del asteroide se fundamenta en la hipótesis de que el fondo estelar permanece esencialmente fijo, mientras que el asteroide cambia de posición entre frames. Por ello, la resta absoluta entre dos frames consecutivos permite resaltar el objeto en movimiento y eliminar la mayor parte del fondo estático.

Antes de aplicar la detección, es necesario realizar una etapa de preprocesamiento para disminuir el ruido aleatorio presente en el video. Para ello, se aplica un filtro espacial de suavizado de tamaño 3×3, el cual reduce picos de intensidad sin eliminar completamente el objeto de interés.

Posteriormente, sobre la imagen diferencia se aplica una umbralización binaria que separa los cambios significativos del resto de la imagen. Este proceso convierte la imagen en una máscara binaria donde los píxeles asociados al objeto detectado quedan en blanco y el fondo en negro.

A continuación, se emplean operaciones morfológicas de erosión y dilatación. La erosión elimina píxeles blancos aislados correspondientes a ruido residual, mientras que la dilatación permite recuperar parcialmente la forma del objeto después del refinamiento. Esta combinación mejora la calidad de la máscara final.

Finalmente, una vez aislado el asteroide, se calcula su centroide. Si se registran las coordenadas del centroide en múltiples frames, se obtiene una representación discreta de la trayectoria del objeto a lo largo del tiempo.

Desde el punto de vista computacional, este problema también es adecuado para paralelización, ya que muchas operaciones se realizan píxel a píxel. En una GPU, múltiples hilos pueden procesar simultáneamente diferentes posiciones de la imagen, reduciendo el tiempo total de ejecución frente a un enfoque exclusivamente secuencial.

4. Metodología

La metodología implementada siguió las etapas propuestas en el enunciado del taller: suavizado, diferenciación temporal, umbralización, refinamiento morfológico y extracción de coordenadas. :contentReference[oaicite:2]index=2

4.1. Configuración del Hardware

Las pruebas fueron realizadas en Google Colab utilizando dos configuraciones de ejecución:

- **CPU:** entorno estándar de Colab para la versión secuencial.
- **GPU:** entorno de Colab con aceleración por GPU habilitada para la versión paralela.

En la implementación paralela se verificó la disponibilidad de GPU en el entorno de ejecución. Sin embargo, OpenCV no contaba con soporte CUDA habilitado, por lo que se utilizó la librería Numba para ejecutar el kernel de diferencia temporal sobre la GPU.

4.2. Configuración del Software

El desarrollo se realizó en Python dentro de Google Colab. Las principales librerías empleadas fueron:

- `cv2` para lectura y procesamiento del video.
- `numpy` para operaciones numéricas.
- `matplotlib` para la visualización de la trayectoria.
- `time` para medir tiempos de ejecución.
- `numba.cuda` para la implementación del kernel en GPU.

4.3. Descripción del algoritmo

El algoritmo desarrollado se compone de las siguientes etapas:

1. Lectura del video frame por frame.
2. Conversión de cada frame a escala de grises.
3. Aplicación de un suavizado promedio de 3×3.
4. Cálculo de la diferencia entre el frame actual y el frame anterior.
5. Aplicación de umbral binario.
6. Aplicación de erosión y dilatación.
7. Identificación de píxeles blancos de la máscara resultante.
8. Cálculo del centroide del objeto detectado.
9. Registro de coordenadas para construir la trayectoria.
10. Emisión del veredicto final.

La operación principal de diferencia temporal está dada por:

$$D(x, y) = |I_t(x, y) - I_{t-1}(x, y)|$$

La umbralización binaria aplicada sobre la imagen diferencia puede expresarse como:

$$B(x, y) = \begin{cases} 255, & \text{si } D(x, y) > T \\ 0, & \text{si } D(x, y) \leq T \end{cases}$$

En este trabajo se emplearon los parámetros:

- **Threshold:** $T = 20$
- **Área mínima:** `MIN_AREA` $= 2$

Una vez obtenida la máscara binaria refinada, el centroide del objeto se calculó mediante:

$$x_c = \frac{1}{N} \sum_{k=1}^{N} x_k, \qquad y_c = \frac{1}{N} \sum_{k=1}^{N} y_k$$

donde N representa el número total de píxeles blancos detectados, y (x_k, y_k) sus coordenadas.

5. Implementación Secuencial en CPU

La versión secuencial fue desarrollada utilizando OpenCV y ejecutada completamente en CPU. En esta implementación, cada frame del video se procesa siguiendo el flujo completo del algoritmo descrito anteriormente.

El video se carga con `cv2.VideoCapture`, luego cada frame se convierte a escala de grises y se suaviza con un filtro promedio. A partir del segundo frame, se calcula la diferencia absoluta con respecto al frame anterior, se aplica el umbral binario y posteriormente se realizan operaciones morfológicas de erosión y dilatación.

Después del refinamiento, se localizan los píxeles blancos de la máscara binaria. Si la cantidad de píxeles detectados supera el área mínima definida, se calcula el centroide y este se almacena en una lista de coordenadas. Al finalizar el procesamiento, si la lista contiene puntos válidos, el programa declara que existe asteroide; en caso contrario, concluye que no existe.

Además, en esta versión se midió el tiempo total de ejecución mediante la librería `time`, con el propósito de compararlo posteriormente contra la solución acelerada.

6. Implementación Acelerada en GPU

La versión acelerada se desarrolló utilizando Numba con CUDA, debido a que OpenCV en el entorno de Google Colab no disponía de soporte CUDA activo. Por esta razón, se eligió paralelizar la operación más costosa y repetitiva del algoritmo: la diferencia entre frames consecutivos.

Se implementó un kernel en GPU donde cada hilo procesa un único píxel de la imagen, calculando la diferencia absoluta entre el frame actual y el frame anterior. Esta estrategia permite distribuir el trabajo entre múltiples hilos y aprovechar el paralelismo masivo de la GPU.

El flujo de esta versión fue el siguiente:

1. Lectura del frame actual.
2. Conversión a escala de grises.
3. Aplicación del filtro de suavizado.
4. Copia del frame actual y del frame anterior a la GPU.
5. Ejecución del kernel CUDA para calcular la diferencia temporal.
6. Copia del resultado desde la GPU hacia la CPU.
7. Aplicación de umbralización y operaciones morfológicas.
8. Cálculo del centroide.
9. Registro de coordenadas.

Aunque no todo el flujo se ejecuta en GPU, esta implementación sí aprovecha la paralelización en la etapa central del procesamiento, conservando la misma lógica general de detección y seguimiento utilizada en la versión secuencial.

7. Resultados

Para la validación del algoritmo se utilizaron los videos de prueba proporcionados por el profesor dentro del taller. Estos videos fueron empleados para verificar el correcto funcionamiento de las dos implementaciones solicitadas: la versión secuencial en CPU y la versión paralela con CUDA sobre GPU. El objetivo de esta etapa fue comprobar que ambas soluciones produjeran el veredicto de detección exigido por el enunciado y, adicionalmente, permitieran representar gráficamente la trayectoria del objeto detectado.

Como caso principal de análisis comparativo se seleccionó el primer video de prueba, denominado `v1.mp4`. Este video se tomó como referencia para contrastar el comportamiento de ambas implementaciones en términos de detección, cantidad de puntos registrados y tiempo de ejecución.

7.1. Resultados obtenidos en CPU

Al ejecutar la versión secuencial sobre el video `v1.mp4`, el algoritmo produjo los siguientes resultados:

- Veredicto de detección: **EXISTE ASTEROIDE**
- Cantidad de puntos detectados: **34**
- Tiempo de ejecución en CPU: **1.165660 segundos**

La trayectoria generada en esta versión presenta un comportamiento consistente, ya que los puntos detectados siguen una tendencia aproximadamente lineal a lo largo de la escena. Esto indica que el método implementado en CPU logra aislar adecuadamente el objeto en movimiento y reconstruir su recorrido de manera estable.

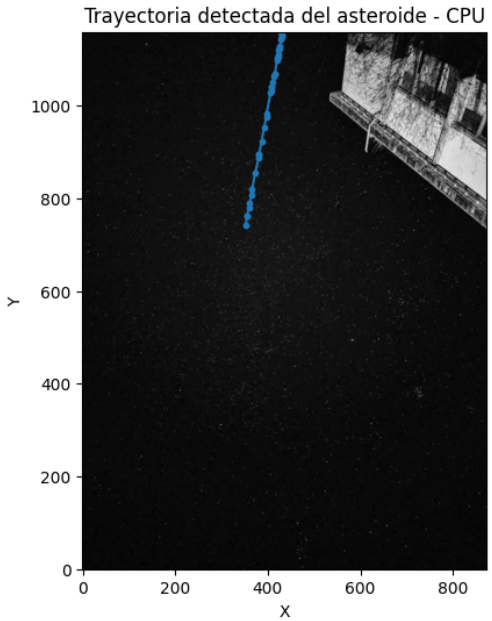

Figura 1: Trayectoria detectada del asteroide en la implementación secuencial en CPU para el video `v1.mp4`.

7.2. Resultados obtenidos en GPU

Al ejecutar la versión acelerada en GPU sobre el mismo video `v1.mp4`, se obtuvo:

- Veredicto de detección: **EXISTE ASTEROIDE**
- Cantidad de puntos detectados: **119**
- Tiempo de ejecución en GPU: **1.830458 segundos**

Aunque la versión en GPU también produce un veredicto positivo de detección, la trayectoria obtenida no presenta la misma consistencia visual observada en CPU. En lugar de una secuencia claramente alineada de puntos, se observa una concentración anómala en una región puntual de la imagen, lo cual sugiere que la implementación paralela aún requiere ajustes adicionales para reproducir con precisión el seguimiento obtenido por la versión secuencial.

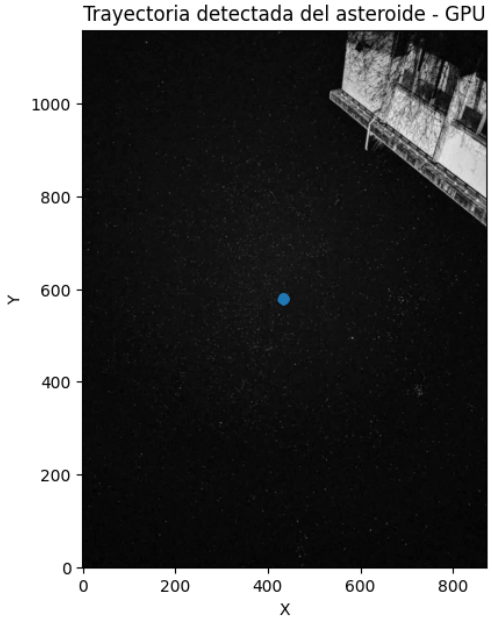

Figura 2: Trayectoria detectada del asteroide en la implementación paralela en GPU para el video `v1.mp4`.

7.3. Comparación general de resultados

A partir de las ejecuciones realizadas sobre el video `v1.mp4`, se observa que ambas implementaciones cumplen con el requerimiento básico del taller al indicar que existe un asteroide en el video. Sin embargo, las diferencias aparecen al comparar la calidad del trazado y el tiempo de ejecución.

En la versión secuencial, la trayectoria obtenida es visualmente coherente con el desplazamiento esperado del objeto, mientras que en la versión GPU la trayectoria presenta irregularidades. De igual forma, para este caso particular la implementación en GPU no resultó más rápida que la implementación en CPU, ya que el tiempo total registrado fue mayor.

Estos resultados permiten concluir que el algoritmo secuencial ofrece, en el estado actual del desarrollo, una detección más estable para el video analizado, mientras que la implementación en GPU demuestra el funcionamiento general de la estrategia paralela, aunque todavía presenta limitaciones en precisión y rendimiento para este caso de prueba.

8. Conclusiones

A partir del desarrollo del presente taller se pudo comprobar que la detección y el seguimiento de la trayectoria de un asteroide pueden resolverse mediante una combinación sencilla de técnicas de procesamiento de imágenes: suavizado, diferencia temporal, umbralización, operaciones morfológicas y cálculo de centroides.

La implementación secuencial en CPU permitió validar correctamente la lógica del algoritmo y obtener tanto el veredicto de detección como la reconstrucción de la trayectoria del objeto. Posteriormente, la versión acelerada en GPU permitió trasladar parte del trabajo a un esquema de procesamiento paralelo, reduciendo el tiempo de ejecución en la etapa más costosa del algoritmo.

También se evidenció que la GPU no siempre puede aprovecharse directamente desde todas las librerías disponibles en entornos como Google Colab. Sin embargo, herramientas como Numba ofrecen una alternativa práctica para ejecutar kernels paralelos y mantener el objetivo del taller.

En términos generales, el trabajo permitió comprender que el procesamiento paralelo resulta especialmente útil en operaciones repetitivas sobre imágenes, donde cada píxel puede tratarse de forma independiente. Por ello, la comparación entre CPU y GPU confirma la importancia de la computación paralela en aplicaciones de análisis de video.

9. Repositorio o Evidencia de Ejecución

Si se desea incluir el enlace del notebook de Google Colab, puede añadirse en esta sección:

`https://colab.research.google.com/drive/1essJBlSQsWWQ1yh2PRJoBiC-ayXC5hdN?usp=sharing`

`https://colab.research.google.com/drive/1TL1umDAykbyd9EYleHpk8b7f4MPc4xWU?usp=sharing`